password parameter, though, and enter it later, as:

```
# vzctl set 101 --userpasswd root:YOURPASSWORD
```

In order to configure an IP address in a VM, you need to select a VM and then click the '*Open VNC console*' link. If the JVM and browser plug-in are present on your PC, you'll see the console shown in Figure 7.

Run *vzlist* to get a list of OpenVZ machines:

```
# /usr/sbin/vzlist
CTID      NPROC STATUS    IP_ADDR         HOSTNAME
       101       198 running   -               liferay_x64
       102        42 running   -               portal_city
       105        66 running   -               Transport
       200        28 running   -               wiki
```

You can also stop, start and reconfigure VMs at the command line with vzctl:

```
# vzctl start 101
```

To save an OpenVZ machine (e.g., VM ID 300) as a single archive file, run *vzdump*:

```
# vzdump --compress --snapshot --dumpdir /var/lib/vzbackup 300
```

Later, you can restore it with *vzrestore* into another VM, say with ID 400:

```
# vzrestore vzdump-openvz-300-2010_02_09-02_00_02.tgz 400
```

QEMU VMs

The second type of virtualisation, QEMU, is full virtualisation, where the CPU, devices, drives and memory are emulated. Therefore, you can use any operating system for the x86/x64 processor (Microsoft Windows, OpenSolaris (x86), QNX or FreeBSD) and not just Linux. Figure 8 shows the Web-based creation of a QEMU VM, much like that for OpenVZ.

Proxmox provides console tools for managing QEMU-based VMs too:

```
# /usr/sbin/qm create 106 --vlan5 rtl8139=1E:0B:BB:A4:94:B9
--cdrom local:iso/windows_server_2008_standard_r2_x64_lic.
iso --name Win2008_Server --bootdisk ide0 --ostype w2k8 --ide0
local:32,format=raw --memory 2048 --onboot yes --sockets 1
```

This creates a QEMU VM with ID 106, a pre-allocated 32 GB IDE disk and 2 GB of RAM. The VM runs a Windows 2008 OS, which will host Microsoft SQL Server 2008.

Please note the parameter '*Disk Storage*' in Figure 8. Its default value is '(*local dir*)', which means that Proxmox will place a single contiguous file, with the filesystem for VM



Figure 8: QEMU container creation is similar to the procedure for OpenVZ

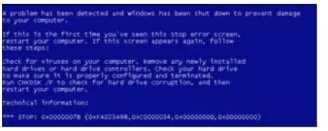


Figure 9: Windows BSOD caused by driver problems

ID 106, in the '*/var/lib/vz/images/106/*' directory. However, we had already decided to reserve '*/var/lib/vz/*' for OpenVZ containers only. The optimal scenario is to go to '*Configuration > Storage*', create a new LVM Group to the storage pool, and later re-use this logical volume as storage for VM ID 106.

Full control over QEMU VMs is available via the Web interface, or by means of console commands qm *. For example, to get a list of registered QEMU containers, run '*qm list*' :

```
# /usr/sbin/qm list
      VMID NAME                 STATUS     MEM(MB)
BOOTDISK(GB) PID
       118 terminal_server      stopped    2048
8.00 0
       132 xp_client_LVM        running    3072
24.00 11534
       500 cacti_KVM            running    512
5.00 11375
```

Migration

An important question that arises every time you change your infrastructure is how to keep all existing projects and transfer them painlessly onto a new platform. In Proxmox' case, this could be done really transparently! You directly copy the virtual containers' image files from VMware, VirtualBox and QEMU; place them on the Proxmox host and power on these transferred VM(s). Some other file formats (used by VirtualPC, Parallels or Bochs) are also supported, but you need to convert them first into RAW format. For example:

```
# qemu-img convert -f vmdk MyOldWindowsServer2003.vmdk -O raw
MigratedWindows2003.raw
```

(Continued on Page 75....)